

UNIT III

GENERATIVE MODELS AND MULTIMODAL AI

Syllabus : Generative Models as Probabilistic Models – Representation Learning and Latent Space Concepts – Autoencoders and Variational Autoencoders (VAE) – Generative Adversarial Networks (GANs) – Diffusion Models and Image Synthesis – Comparative Analysis of Generative Architectures – Evolution of Multimodal AI Systems – Text, Image and Audio Understanding – Emerging Multimodal Foundation Models.

GENERATIVE MODELS AS PROBABILISTIC MODELS

Generative Artificial Intelligence aims to create new data that resembles the data on which it has been trained. Unlike discriminative models, which learn only the relationship between inputs and outputs for prediction or classification, generative models attempt to learn the underlying probability distribution of the training data. Once this distribution has been learned, the model can generate entirely new samples that follow the same statistical characteristics as the original dataset.

At the core of every generative model lies probability theory. Since real-world data contains uncertainty, randomness, and multiple possible outcomes, generative models represent data using probabilistic distributions rather than deterministic rules. Instead of memorizing training examples, they estimate the probability of observing particular data samples and use this learned probability distribution to generate new instances.

Therefore, generative models are fundamentally probabilistic models, where learning is based on estimating probability distributions from observed data.

Real-world data is inherently uncertain. A single input may correspond to multiple valid outputs, and the exact outcome cannot always be determined using fixed mathematical rules.

Consider a language model predicting the next word in a sentence. Given the input sequence, there may be several grammatically and semantically correct continuations. Rather than selecting a single deterministic output, the model estimates the probability of every possible next token and generates one based on the learned probability distribution.

Similarly, in image generation, there are numerous valid images corresponding to a given text prompt. The model must therefore learn the distribution of possible images rather than a single fixed image.

Consequently, generative models are designed to answer the following probabilistic question:

"Given the observed training data, what is the probability that a new sample belongs to the same distribution?"

This probabilistic formulation enables the generation of diverse and realistic outputs.

Probability Theory in Generative Models

Probability provides a mathematical framework for representing uncertainty.

Suppose X represents a random variable corresponding to data samples.

The probability of observing a particular sample is represented as

$$P(X = x)$$

where

- X is the random variable,

- x is one possible observation.

Rather than storing every training example, a generative model learns the probability distribution

$$P(X)$$

Once this distribution has been estimated, the model can generate new samples by drawing observations from the learned probability distribution.

The overall objective becomes

$$\text{Learn } P(X)$$

instead of memorizing

$$X_1, X_2, \dots, X_n$$

Random Variables

A random variable is a mathematical variable whose value is determined by a probabilistic process.

Instead of always producing the same outcome, a random variable may assume different values according to an associated probability distribution.

A random variable is commonly denoted by an uppercase letter such as X while its observed values are written as x

If

$$X = \text{Image}$$

Then x may represent one particular image.

Similarly,
if

$$X = \text{Sentence}$$

Then x represents one observed sentence from the dataset.

Generative models attempt to learn the probability distribution governing the random variable.

Probability Distribution

A probability distribution describes how likely different values of a random variable are.

For a discrete random variable, the probability distribution satisfies

$$\sum_x P(x) = 1$$

This indicates that the total probability across all possible observations equals one.

For continuous variables,

the probability density function satisfies

$$\int P(x) dx = 1$$

The distribution completely characterizes the statistical behaviour of the data.

Rather than learning individual samples,

generative models estimate

$$P(x)$$

for every possible observation.

Discrete Probability Distribution

When data consists of countable outcomes, the distribution is called a discrete probability distribution.

Suppose X takes values

$$x_1, x_2, \dots, x_n$$

The probabilities satisfy

$$\sum_{i=1}^n P(x_i) = 1$$

Every possible observation has an associated probability.

Language models frequently use discrete probability distributions because vocabulary tokens are discrete entities.

For example,

the probability of predicting the next token is represented as

$$P(w_i)$$

for every vocabulary token

$$w_i.$$

Continuous Probability Distribution

Many generative models operate in continuous spaces. Instead of assigning probabilities to individual values, they estimate a probability density function (PDF).

The probability of observing values within an interval $[a, b]$

is

$$P(a \leq X \leq b) = \int_a^b P(x) dx$$

Diffusion models,

Variational Autoencoders,

and normalizing flows commonly learn continuous probability distributions.

Joint Probability Distribution

Generative models rarely operate on individual variables. Instead, they model the probability of observing multiple variables simultaneously.

Suppose X and Y represent two random variables.

Their joint probability is $P(X, Y)$. This distribution describes the probability that both variables occur together.

For example,

an image generation model may consider

- object appearance,
- background,
- lighting,

simultaneously.

The complete data distribution becomes

$$P(X_1, X_2, \dots, X_n)$$

Modern generative models attempt to estimate this joint probability distribution over all variables.

Marginal Probability

Sometimes, the probability of only one variable is required. This is called the marginal probability.

Suppose
the joint distribution is

$$P(X, Y)$$

The marginal distribution of

$$X$$

is obtained by summing over all possible values of

$$P(X) = \sum_Y P(X, Y)$$

For continuous variables,

$$P(X) = \int P(X, Y) dY$$

Marginalization removes unwanted variables while preserving the probability of the variable of interest.

Many latent-variable generative models use marginalization extensively during training.

Conditional Probability

Conditional probability measures the probability of one variable given that another variable is already known.

Mathematically,

$$P(Y|X) = \frac{P(X, Y)}{P(X)}$$

This equation describes how the probability of Y changes when X has already been observed.

Many generative models generate data sequentially by repeatedly estimating conditional probabilities.

For example,

autoregressive language models estimate

$$P(x_t | x_1, x_2, \dots, x_{t-1})$$

which represents the probability of the next token given all previous tokens.

The complete sequence probability becomes

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t | x_1, \dots, x_{t-1})$$

This formulation forms the mathematical basis of Transformer-based language models such as GPT.

Bayes' Theorem

One of the most important equations in probabilistic modelling is Bayes' Theorem. It relates conditional probabilities in opposite directions. The theorem is expressed as:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Bayes' theorem enables probabilistic inference. Suppose Y represents a hidden variable, and X represents observed data. Rather than directly computing

$$P(Y|X),$$

which may be difficult, the model computes

- likelihood

$$P(X|Y)$$

- prior probability

$$P(Y)$$

- evidence

$$P(X)$$

The posterior probability becomes

$$P(Y|X) = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

Bayesian inference forms the theoretical foundation of many probabilistic generative models, particularly Variational Autoencoders.

Prior Distribution

Before observing any data, a probabilistic model assumes an initial probability distribution called the prior distribution. It is represented as

$$P(z)$$

where

z is a latent variable.

The prior expresses the model's assumptions before observing training data.

Many deep generative models assume

$$P(z) = N(0, I)$$

which represents a standard multivariate Gaussian distribution.

Likelihood Function

The likelihood measures how probable the observed data is for a given latent representation.

It is represented as

$$P(x|z)$$

where

- x is observed data,
- z is the latent variable.

The likelihood describes the data generation process.
Learning an accurate likelihood function enables the model to reconstruct realistic observations from latent variables.

Posterior Distribution

After observing data,
the probability distribution over latent variables becomes the posterior distribution.
It is represented as

$$P(z|x)$$

Using Bayes' theorem,

$$P(z|x) = \frac{P(x|z)P(z)}{P(x)}$$

The posterior combines

- prior knowledge,
- observed evidence,

to estimate the hidden variables responsible for generating the observed data.

Posterior inference is central to Bayesian generative models and Variational Autoencoders.

General Mathematical Objective of Generative Models

GENERAL MATHEMATICAL OBJECTIVE OF GENERATIVE MODELS

The primary objective of a generative model is not merely to reproduce the training data but to learn the probability distribution from which the data originates.

Suppose the training dataset consists of samples

$$x_1, x_2, \dots, x_n$$

Rather than storing these samples individually, the model estimates

$$P(x)$$

or

$$P(x, z)$$

where

.z represents latent variables.

Once this distribution has been learned,

new samples can be generated by sampling from the learned probability distribution.

For latent-variable models,

the generation process is

$$z \sim P(z)$$

followed by

$$x \sim P(x|z)$$

This probabilistic sampling mechanism allows the model to generate infinitely many new observations that follow the statistical characteristics of the training data rather than duplicating existing examples.

The fundamental objective of probabilistic generative modelling is to estimate the true

data distribution

$$P_{\text{data}}(x)$$

using a parameterized model

$$P_{\theta}(x)$$

Where θ represents the learnable parameters.

Training attempts to satisfy

$$P_{\theta}(x) \approx P_{\text{data}}(x)$$

As the learned distribution approaches the true data distribution, the generated samples increasingly resemble real-world observations.

MATHEMATICAL FORMULATION OF GENERATIVE MODELS

A generative model is a mathematical model that attempts to learn the probability distribution responsible for producing the observed training data. Let the observed dataset be represented by

$$D = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\},$$

where each observation

$$x^{(i)} \in X$$

belongs to a data space X . The data space may contain token sequences, images, audio signals, video frames or continuous feature vectors.

The observations are assumed to be drawn independently from an unknown real-world distribution:

$$x^{(i)} \sim p_{\text{data}}(x).$$

The true distribution $p_{\text{data}}(x)$ is not directly available. Only a finite collection of samples drawn from it is observable. The objective of generative modelling is therefore to construct a parameterised distribution

$$p_{\theta}(x),$$

where θ represents the learnable parameters of the model, such that

$$p_{\theta}(x) \approx p_{\text{data}}(x).$$

Once $p_{\theta}(x)$ has been learned, the model can generate a new observation by sampling

$$\tilde{x} \sim p_{\theta}(x).$$

The generated observation $\tilde{x}$ is not necessarily present in the original training dataset. However, it should exhibit statistical, structural and semantic characteristics consistent with the data distribution.

Data Distribution

The data distribution represents the complete probability law governing the observations. For discrete data, $p_{\text{data}}(x)$ is a probability mass function satisfying

$$p_{\text{data}}(x) \geq 0$$

and

$$\sum_{\mathbf{x} \in \mathcal{X}} p_{\text{data}}(\mathbf{x}) = 1.$$

For continuous data, $p_{\text{data}}(\mathbf{x})$ is a probability density function satisfying

$$p_{\text{data}}(\mathbf{x}) \geq 0$$

and

$$\int_{\mathcal{X}} p_{\text{data}}(\mathbf{x}) d\mathbf{x} = 1.$$

The probability assigned to an exact continuous observation is technically zero. Probabilities are instead obtained by integrating the density over a region $A \subseteq \mathcal{X}$:

$$P(\mathbf{x} \in A) = \int_A p_{\text{data}}(\mathbf{x}) d\mathbf{x}.$$

A generative model must reproduce not only common observations but also the dependencies among the components of each observation. For a multidimensional random vector

$$\mathbf{x} = [x_1, x_2, \dots, x_d]^T,$$

the model must approximate the joint distribution

$$p_{\text{data}}(x_1, x_2, \dots, x_d).$$

Learning this high-dimensional joint distribution is the central mathematical problem of generative modelling.

Maximum-Likelihood Formulation

The likelihood of the complete training dataset under the model is

$$p_{\theta}(D) = \prod_{i=1}^n p_{\theta}(x^{(i)}),$$

assuming that observations are independently and identically distributed.

Maximum-likelihood estimation determines the parameter value that makes the observed dataset most probable:

$$\theta_{\text{MLE}} = \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(x^{(i)}).$$

Products of many probabilities may become numerically extremely small. The logarithm is therefore applied:

$$\theta_{\text{MLE}} = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(x^{(i)}).$$

Equivalently, training may minimise the negative log-likelihood:

$$L_{\text{NLL}}(\theta) = -\sum_{i=1}^n \log p_{\theta}(x^{(i)}).$$

The average negative log-likelihood is

$$L_{\text{NLL}}(\theta) = -\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(x^{(i)}).$$

The negative log-likelihood provides a direct probabilistic training objective for models in which the density $p_{\theta}(x)$ can be computed or approximated.

TYPES OF PROBABILISTIC GENERATIVE MODELS

Probabilistic generative models can be classified according to how they represent the data distribution, how their likelihood is evaluated and how new samples are generated. The major categories include explicit density models, implicit density models, latent-variable models, autoregressive models and energy-based models.

These categories are not always mutually exclusive. A single architecture may belong to more than one category. For example, an autoregressive model is an explicit-density model, while a variational autoencoder is both an explicit-density model and a latent-variable model.

1) EXPLICIT DENSITY MODELS

An explicit density model defines a probability distribution $p_{\theta}(x)$ in an analytically specified form. The model provides either an exact expression for the data likelihood or a tractable approximation or lower bound to it.

The defining characteristic is that the model explicitly represents the probability density: $p_{\theta}(x)$.

For discrete data, this corresponds to a probability mass function. For continuous data, it corresponds to a probability density function.

Explicit density models are commonly divided into tractable-density models and approximate-density models.

Tractable Explicit Density Models

A tractable explicit-density model allows the likelihood $p_{\theta}(x)$

to be evaluated exactly or efficiently.

The model may be trained directly using maximum likelihood:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(x^{(i)}).$$

Autoregressive models and normalising-flow models are important members of this category.

In an autoregressive model, the joint distribution is decomposed into a product of conditional distributions. In a normalising flow, a simple base distribution is transformed through a sequence of invertible mappings, while the exact density is computed through the change-of-variables formula.

The principal advantage of tractable explicit models is that the likelihood can be evaluated directly. This permits statistically principled training and quantitative comparison. Their limitation is that architectural constraints are often required to preserve likelihood tractability.

Approximate Explicit Density Models

Some generative models define a probability distribution explicitly but make exact likelihood evaluation computationally intractable. These models therefore optimise an approximation or bound.

A latent-variable model defines

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z}).$$

The marginal likelihood is

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z}) d\mathbf{z}.$$

Although this equation explicitly defines $p_{\theta}(\mathbf{x})$, the integral may not possess a closed-form solution. The model then uses approximate inference, variational optimisation, Monte Carlo estimation or other numerical techniques.

Variational autoencoders belong to this category because they optimise a lower bound on the log-likelihood rather than evaluating the exact marginal likelihood directly.

Characteristics of Explicit Density Models

Explicit models establish a mathematically defined probability law over observations. As a result, they can assign likelihood values to data, support maximum-likelihood-based training and provide a direct probabilistic interpretation.

However, a high likelihood does not always guarantee that generated samples will possess the highest perceptual quality. Furthermore, the requirement to maintain a valid and computable density can constrain the model structure.

2) IMPLICIT DENSITY MODELS

An implicit generative model does not provide an analytically tractable expression for $p_{\theta}(\mathbf{x})$.

Instead, it defines a stochastic procedure that produces samples from the model distribution.

The generation process is generally expressed as

$$\mathbf{z} \sim p(\mathbf{z}),$$

followed by

$$\mathbf{x} = G_{\theta}(\mathbf{z}),$$

where G_{θ} is a deterministic neural generator and $\mathbf{z}$ is a random latent variable.

This transformation induces a distribution over $\mathbf{x}$:

$$\mathbf{x} \sim p_{\theta}(\mathbf{x}),$$

but the numerical density value $p_{\theta}(\mathbf{x})$ may not be directly available.

The model therefore defines the distribution through its sampling mechanism rather than through a closed-form density.

Training Implicit Models

Since an explicit likelihood is unavailable, implicit models cannot ordinarily be trained by directly minimising negative log-likelihood. Instead, they use alternative distribution-matching objectives.

Adversarial training introduces a discriminator D_ϕ that attempts to distinguish real samples from generated samples. The generator G_θ attempts to produce samples that the discriminator cannot distinguish from real data.

The adversarial objective is

$$\min_{\theta} \max_{\phi} \left[E_{x \sim p_{\text{data}}} \log D_{\phi}(x) + E_{z \sim p(z)} \log (1 - D_{\phi}(G_{\theta}(z))) \right].$$

At the ideal equilibrium,

$$p_{\theta}(x) = p_{\text{data}}(x).$$

Other implicit models may use moment matching, integral probability metrics, optimal transport distances or kernel-based discrepancies.

Properties of Implicit Models

Implicit models provide considerable architectural flexibility because they do not require a normalised density function. Their generators can transform random vectors into highly structured observations through arbitrary differentiable networks.

This flexibility can support high-quality sample generation. However, the absence of a tractable likelihood makes probabilistic evaluation more difficult. Training may also suffer from distributional collapse, unstable optimisation or incomplete coverage of the data distribution.

Generative adversarial networks are the principal example of implicit density models.

3) LATENT VARIABLE MODELS

A latent-variable model assumes that observed data is generated through unobserved random variables.

Let z denote a latent variable and x denote an observed variable. Let z denote a latent variable and x denote an observed variable. The generative process is described by

$$z \sim p(z)$$

and

$$x \sim p_{\theta}(x|z).$$

The joint probability distribution is

$$p_{\theta}(x, z) = p_{\theta}(x|z)p(z).$$

The prior $p(z)$ specifies the probability distribution over latent representations. The conditional distribution $p_{\theta}(x|z)$ defines how observations are generated from those representations.

Marginal Likelihood

Because z is unobserved, the probability of an observation must be obtained by marginalising the latent variable:

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz.$$

Substituting the joint factorisation gives

$$p_{\theta}(x) = \int p_{\theta}(x|z)p(z) dz.$$

For discrete latent variables, the integral is replaced by a sum:

$$p_{\theta}(\mathbf{x}) = \sum_z p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z}).$$

The marginal likelihood combines all possible latent explanations of the observation, weighted by their prior probabilities.

Posterior Inference

After an observation $\mathbf{x}$ has been obtained, the distribution over latent variables is given by Bayes' theorem:

$$p_{\theta}(\mathbf{z}|\mathbf{x}) = \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_{\theta}(\mathbf{x})}.$$

The posterior identifies latent configurations that plausibly explain the observation. The denominator is

$$p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z}) d\mathbf{z}.$$

For high-dimensional neural models, this integral is commonly intractable. Exact posterior inference is therefore unavailable, motivating approximate-inference techniques.

Variational Inference

Variational inference introduces a tractable approximate posterior

$$q_{\phi}(\mathbf{z}|\mathbf{x}),$$

parameterised by ϕ .

The objective is to make $q_{\phi}(\mathbf{z}|\mathbf{x})$ approximate the true posterior:

$$q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x}).$$

The log marginal likelihood can be written as

$$\log p_{\theta}(\mathbf{x}) = L_{\text{ELBO}}(\theta, \phi; \mathbf{x}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}|\mathbf{x})).$$

Because KL divergence is non-negative,

$$D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}|\mathbf{x})) \geq 0,$$

it follows that

$$\log p_{\theta}(\mathbf{x}) \geq L_{\text{ELBO}}(\theta, \phi; \mathbf{x}).$$

The evidence lower bound is

$$L_{\text{ELBO}} = E_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})).$$

The first term measures the expected log-likelihood of reconstructing the observation from the latent variable. The second term regularises the approximate posterior towards the prior distribution.

Training maximises

$$L_{\text{ELBO}}$$

or equivalently minimises

$$-L_{\text{ELBO}}.$$

REPRESENTATION LEARNING AND LATENT SPACE CONCEPTS

Representation learning is a branch of machine learning concerned with automatically discovering useful ways of representing raw data. Instead of manually defining features, the learning algorithm identifies internal features that make prediction, classification, generation, clustering, or reconstruction easier.

Real-world data is usually available in a raw and high-dimensional form. An image is represented through pixel values, text is represented as tokens, speech is represented as a sequence of signal amplitudes, and industrial sensor data is represented through numerical measurements over time. These raw observations may not directly expose meaningful information.

For example, in an image-recognition problem, the raw pixels do not explicitly specify concepts such as edges, shapes, texture, object parts, pose, lighting, or object identity. A representation-learning model automatically converts the raw input into an internal mathematical form that captures such useful characteristics.

Traditional Machine Learning

Input → Manual Feature Extraction → Feature Vector → Classifier → Output

Representation Learning

Input → Encoder → Latent Representation → Classifier → Output

Mathematically,

$$\hat{y} = g\phi(f\theta(x))$$

where

- $f\theta$ learns the representation.
- $g\phi$ performs the prediction.

The model learns the optimal parameters by minimizing a loss function

$$(\theta, \phi) = \arg \min L(g\phi(f\theta(x)), y)$$

where L represents the loss function between the predicted output and the true output.

A good representation should preserve important information while removing irrelevant details such as noise.

If two samples belong to the same class, their latent representations should be close:

$$d(z_i, z_j) \approx 0$$

If they belong to different classes, their representations should be farther apart:

$$d(z_i, z_k) > d(z_i, z_j)$$

where $d(\cdot)$ denotes a distance measure such as Euclidean distance.

An effective representation should have the following properties:

- Informative – preserves useful information.
- Robust – resistant to noise.
- Generalizable – performs well on unseen data.
- Structured – maintains meaningful relationships.
- Transferable – reusable for multiple tasks.

Representation of Data

A representation is a mathematical description of data used internally by a machine-learning model.

Suppose an image contains D pixel values. It can be represented as a vector:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \end{bmatrix} \in \mathbb{R}^D$$

where:

- x is the input vector.
- $x_1, x_2, \dots, x_D$ are the individual pixel values.
- $\mathbb{R}^D$ denotes a D -dimensional real-valued vector space.

The representation-learning model transforms this input into another vector called the latent representation:

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_d \end{bmatrix} \in \mathbb{R}^d$$

where:

- z is the learned representation (latent vector).
- $z_1, z_2, \dots, z_d$ are the learned features.
- $\mathbb{R}^d$ denotes a d -dimensional latent space.

The dimension d may be smaller than, equal to, or greater than the original dimension D .

Case 1: Compression

If

$$d < D$$

then the representation is a compressed representation of the original data.

Case 2: Expansion

If

$$d > D$$

then the representation expands the input into a higher-dimensional feature space where useful patterns may become easier to separate.

Hence, representation learning is not merely dimensionality reduction. Its primary objective is to learn a feature representation that captures meaningful information for downstream tasks such as classification, prediction, clustering, or generation.

Representation Learning and Feature Engineering

In traditional machine learning, features are manually created by domain experts. For example, an image-recognition system may use manually selected features such as edge density, shape descriptors, colour histograms, and texture measurements.

The traditional workflow is

$$x \xrightarrow{\text{Manual Feature Extraction}} h \xrightarrow{\text{Classifier}} \hat{y}$$

where:

- x = Input data
- h = Hand-crafted feature vector
- $\hat{y}$ = Predicted output

Here, h is a manually engineered feature vector.

In representation learning, the feature extractor is learned automatically.

$$x \xrightarrow{f_{\theta}} z \xrightarrow{g_{\phi}} y$$

where

- f_{θ} = Encoder (representation-learning function)
- g_{ϕ} = Prediction model (classifier/regressor)
- z = Learned representation (latent feature vector)

LATENT SPACE CONCEPTS

Latent space is one of the most fundamental concepts in Deep Learning and Generative AI. It refers to a lower-dimensional feature space where a machine learning model stores meaningful representations of input data. Instead of working directly with high-dimensional raw data such as images, text, or audio, the model transforms them into compact numerical vectors called latent representations. These vectors capture the essential characteristics of the data while removing redundant or irrelevant information. For example, a 28×28 grayscale image contains 784 pixel values. Rather than processing all 784 values directly, an autoencoder may compress the image into a 32-dimensional latent vector that still preserves the important visual features required for reconstruction or classification.

Latent Variables

A latent variable is a hidden variable that cannot be directly observed but influences the observed data. In real-world datasets, many important factors are not explicitly available. For example, in a face image, factors such as identity, facial expression, age, lighting conditions, and viewing angle are hidden variables that determine the appearance of the image.

A generative model assumes that the latent variable is generated first and then produces the observed data.

$$z \sim p(z)$$

$$x \sim p(x | z)$$

where

- x – Observed data
- z – Latent (hidden) variable
- $p(z)$ – Prior distribution of latent variables
- $p(x|z)$ – Probability of generating data from the latent variable

Thus, latent variables provide a compact representation of the hidden structure of the data.

Latent Representation

A latent representation is the numerical vector produced by the encoder after processing the input.

The encoder learns the mapping

$$z = f_{\theta}(x)$$

where

- x = Input data
- z = Latent representation
- f_{θ} = Encoder network

The latent representation contains the most informative features of the input while ignoring unnecessary details such as noise or redundancy.

For example,

- Original image: 784 features
- Latent representation: 32 features

Even though the latent vector has much lower dimensionality, it still captures the important characteristics of the original image.

Latent Space

The collection of all latent representations produced by a model is called the latent space. Each input sample is represented as a point in this space.

If the latent vector has d dimensions, then the latent space is represented as

$$\mathbf{Z} \subseteq \mathbb{R}^d$$

Each input is mapped as

$$\mathbf{x} \rightarrow \mathbf{z}$$

Similar inputs are placed close together in latent space, while dissimilar inputs are placed farther apart.

For example, in a handwritten digit recognition system:

- Images of the digit **0** form one cluster.
- Images of **1** form another cluster.
- Similar digits occupy nearby regions.

This organization makes learning and classification much easier.

Geometry of Latent Space

The geometry of latent space determines how learned representations are organized. The similarity between two latent vectors is commonly measured using the Euclidean distance.

$$d(\mathbf{z}_i, \mathbf{z}_j) = \|\mathbf{z}_i - \mathbf{z}_j\|_2$$

A smaller distance indicates greater similarity between the samples.

Another commonly used similarity measure is Cosine Similarity, especially in Natural Language Processing.

Cosine Similarity measures the angle between two vectors and ranges from -1 to 1 .

- 1 → Very similar vectors
- 0 → No similarity
- -1 → Opposite vectors

Text embedding models such as BERT and Sentence Transformers commonly use cosine similarity for semantic search.

Latent Space Operations

One of the most interesting properties of latent space is that meaningful operations can be performed directly on latent vectors.

1. Interpolation

Interpolation generates smooth transitions between two latent vectors.

If two latent vectors represent different faces, intermediate latent vectors produce gradual changes between the two faces. This property is widely used in image morphing and generative models.

2. Latent Vector Arithmetic

Latent vectors often encode semantic relationships.

A famous example is:

King – Man + Woman $\approx$ Queen

This demonstrates that latent space captures meaningful relationships between concepts rather than simply storing numerical values.

Characteristics of a Latent Space

An effective latent space should possess the following properties:

- **Compactness:** Represents data using fewer dimensions.
- **Semantic Structure:** Similar data points are located close together.
- **Continuity:** Small changes in latent vectors produce smooth changes in the generated output.
- **Generalization:** Unseen data can also be represented meaningfully.
- **Transferability:** Learned representations can be reused for different tasks such as classification, retrieval, clustering, and generation.

Latent Space Operations

One of the properties of latent space is that meaningful operations can be performed directly on latent vectors.

1. Interpolation

Interpolation generates smooth transitions between two latent vectors.

If two latent vectors represent different faces, intermediate latent vectors produce gradual changes between the two faces. This property is widely used in image morphing and generative models.

2. Latent Vector Arithmetic

Latent vectors often encode semantic relationships.

A famous example is: $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$

This demonstrates that latent space captures meaningful relationships between concepts rather than simply storing numerical values.

Applications of Latent Space

Latent space is widely used in modern Artificial Intelligence systems.

Some important applications include:

- Image Generation (GANs and Diffusion Models)
- Image Compression
- Face Recognition
- Recommendation Systems
- Semantic Search
- Speech Recognition
- Medical Image Analysis
- Natural Language Processing
- Large Language Models (LLMs)
- Retrieval-Augmented Generation (RAG)
- Anomaly Detection

Advantages of Latent Space

- Reduces the dimensionality of complex data.
- Removes redundant and noisy information.
- Improves computational efficiency.
- Captures hidden semantic relationships.
- Enables clustering, interpolation, and data generation.
- Supports transfer learning across multiple AI tasks.

AUTOENCODERS AND VARIATIONAL AUTOENCODERS

Autoencoders and Variational Autoencoders are neural-network architectures used to learn meaningful representations of data.

Both models contain two main components:

1. An encoder, which converts the input into a latent representation.
2. A decoder, which reconstructs or generates data from the latent representation.

The major difference is that a normal Autoencoder maps an input to a fixed latent vector, whereas a Variational Autoencoder maps the input to a probability distribution in latent space.

An Autoencoder is mainly designed for:

- Representation learning
- Dimensionality reduction
- Data compression
- Noise removal
- Feature extraction
- Anomaly detection

A Variational Autoencoder is mainly designed for:

- Generating new data
- Learning a smooth latent space
- Data interpolation
- Image synthesis
- Representation learning
- Probabilistic modelling

AUTOENCODERS

An Autoencoder is an unsupervised neural network that learns to reconstruct its input. The input itself acts as the target output.

For example, when an image is given to an Autoencoder, the network compresses the image into a smaller internal representation and then attempts to reconstruct the original image.

The general process is:

Input → Encoder → Latent Representation → Decoder → Reconstructed Input

The model learns by comparing the reconstructed output with the original input.

If the reconstruction is poor, the parameters of the encoder and decoder are updated. After repeated training, the model learns to preserve the most important information needed to reconstruct the data.

Autoencoder Architecture

Components of an Autoencoder

Input Layer

The input layer receives the original data.

The input may be:

- An image
- A text embedding
- An audio signal
- Sensor readings
- Tabular data

- Medical data
- Manufacturing-process data

For example, a grayscale image of size 28×28 contains:

$28 \times 28 = 784$ pixel values

Therefore, the input is represented as a vector containing 784 values.

Encoder

The encoder transforms the original input into a latent representation.

The encoder operation is represented as $z = f_{\theta}(x)$

where:

- x is the original input.
- f_{θ} is the encoder function.
- θ represents the encoder parameters.
- z is the latent representation.

A simple encoder layer may perform: $z = \sigma(Wx + b)$

where:

- W is the weight matrix.
- b is the bias vector.
- σ is an activation function.
- z is the encoded representation.

The encoder gradually reduces the dimensionality of the input.

For example:

Input layer : 784 neurons

Hidden layer 1 : 256 neurons

Hidden layer 2 : 128 neurons

Latent layer : 32 neurons

Therefore, the original 784-dimensional image is represented using only 32 latent features.

Bottleneck or Latent Representation

The bottleneck is the smallest or most constrained part of the network. It contains the encoded representation of the input.

Original image: 784 values will convert to Latent representation: 32 values

Because the bottleneck cannot store every original value directly, the model must learn the most important patterns.

For an image, the latent vector may capture information such as:

- Shape
- Edges
- Object structure
- Texture
- Orientation
- Colour
- Style
- Object category

The individual latent dimensions may not always have a direct human-readable meaning. However, together they provide a compact mathematical representation of the input.

Decoder

The decoder converts the latent representation back into the original data space.

The decoding operation is:

$$\hat{x} = g\varphi(z)$$

where:

- z is the latent representation.
- $g\varphi$ is the decoder.
- φ represents decoder parameters.
- $\hat{x}$ is the reconstructed output.

The complete Autoencoder is therefore:

$$\hat{x} = g\varphi(f\theta(x))$$

The decoder usually reverses the architecture of the encoder.

Latent layer : 32 neurons

Hidden layer 1 : 128 neurons

Hidden layer 2 : 256 neurons

Output layer : 784 neurons

The final output is reshaped into the original image dimensions.

WORKING OF AN AUTOENCODER

The operation of an Autoencoder can be explained in five steps.

Step 1: Input is provided

Suppose a handwritten digit image is given to the network.

The image has a size of 28×28 pixels and is converted into a vector of 784 pixel values.

Input image is $[0.0, 0.0, 0.2, 0.8, \dots, 0.0]$

Step 2: Encoder extracts important information

The encoder passes the input through several neural-network layers.

Each layer gradually transforms and compresses the input.

784 dimensions



256 dimensions



128 dimensions



32-dimensional latent vector

The final 32-dimensional vector stores the important features of the image.

Step 3: Latent representation is created

The encoder produces a latent vector such as:

$$z = [0.82, -0.15, 0.44, 0.67, \dots, 0.21]$$

This vector is a compressed internal representation of the input image.

It does not store the image pixel by pixel. Instead, it stores patterns that allow the decoder to reconstruct the image.

Step 4: Decoder reconstructs the input

The decoder takes the latent vector and expands it back to the original dimension.

32 dimensions to 128 dimensions to 256 dimensions to 784-dimensional reconstruction

The output is reshaped into a 28×28 image.

Step 5: Reconstruction error is calculated

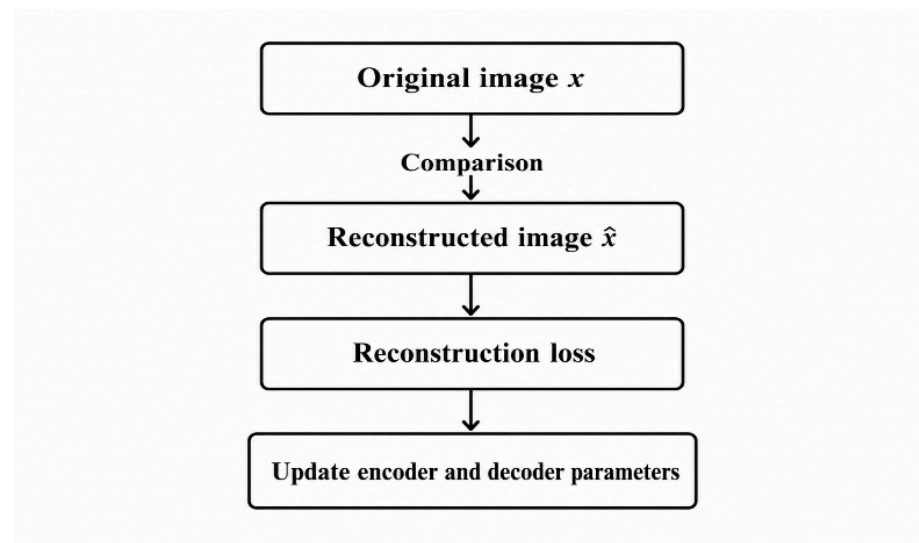
The reconstructed image is compared with the original image.

The difference between them is called the reconstruction error.

For continuous data, Mean Squared Error is commonly used:

$$L_{\text{rec}} = (1/D) \sum (x_j - \hat{x}_j)^2$$

The objective is to minimize the difference between the original input and reconstructed output.



6. Training an Autoencoder

An Autoencoder is trained using backpropagation and gradient descent.

The training process is:

1. Pass input through the encoder.
2. Generate the latent representation.
3. Pass the latent representation through the decoder.
4. Produce the reconstructed output.
5. Calculate reconstruction loss.
6. Backpropagate the loss.
7. Update encoder and decoder weights.
8. Repeat for many training examples.

The general training objective is: Minimize $L(x, \hat{x})$

The model does not need separate class labels because the target is the input itself.

For an input: x

the target output is also: x

The model prediction is: $\hat{x}$

If an Autoencoder has enough capacity, it may learn an identity function: $\hat{x} = x$

This means that the model may copy the input without learning useful features.

To prevent this, constraints are introduced, such as:

- A small bottleneck
- Sparse activations
- Input corruption
- Weight regularization
- Noise injection
- Limited network capacity

These constraints force the network to learn meaningful patterns instead of memorizing each input.

TYPES OF AUTOENCODERS

1) Undercomplete Autoencoder

An Undercomplete Autoencoder has a latent dimension smaller than the input dimension.

Latent dimension < Input dimension

Example:

Input : 784 dimensions

Latent code : 32 dimensions

Output : 784 dimensions

The small bottleneck forces the network to compress the information.

Advantages

- Learns compact representations
- Reduces dimensionality
- Removes redundant information
- Useful for visualization and feature extraction

Limitation

A very small latent space may remove useful information and produce poor reconstruction.

2) Sparse Autoencoder

A Sparse Autoencoder may have a large latent layer, but only a small number of neurons are allowed to be active for each input.

Example:

Latent representation:

[0, 0, 0.82, 0, 0, 0.35, 0, 0, 0.61, 0]

Only a few values are active.

Sparsity encourages different neurons to learn different features.

For an image dataset:

- One neuron may detect vertical edges.
- Another may detect circular shapes.
- Another may detect texture.
- Another may respond to brightness.

Advantages

- Improves feature specialization
- Can improve interpretability
- Reduces feature redundancy

3) Contractive Autoencoder

A Contractive Autoencoder is trained to make its latent representation insensitive to small input changes.

For example, two images that differ only slightly in brightness should produce similar latent representations.

Original image *Slightly modified image*



Latent vector z_1



Latent vector z_2

$$z_1 \approx z_2$$

This improves the robustness of the learned representation.

APPLICATIONS OF AUTOENCODERS

Dimensionality Reduction

An Autoencoder can reduce high-dimensional data into a lower-dimensional

representation.

Unlike PCA, which learns only linear relationships, Autoencoders can learn nonlinear relationships.

Data Compression

The encoder compresses the input into a smaller latent representation.

However, general-purpose compression algorithms may be more suitable when lossless reconstruction is required.

Noise Removal

Denoising Autoencoders remove noise from images, audio, and sensor signals.

LIMITATIONS OF ORDINARY AUTOENCODERS

Although Autoencoders learn useful representations, they have several limitations.

Unstructured latent space

The latent vectors may be scattered irregularly.

Nearby points in latent space may not always produce similar outputs.

Poor generation

Randomly sampling a latent vector may produce an invalid or meaningless output.

Overfitting

A powerful network may memorize training samples.

Deterministic encoding

Each input is mapped to one fixed latent vector.

Discontinuity

Empty gaps may exist between groups of latent points, making smooth interpolation difficult.

These limitations motivated the development of Variational Autoencoders.

VARIATIONAL AUTOENCODERS

A Variational Autoencoder is a probabilistic generative model that learns a structured and continuous latent space.

Unlike a standard Autoencoder, a VAE does not encode an input into one fixed latent vector.

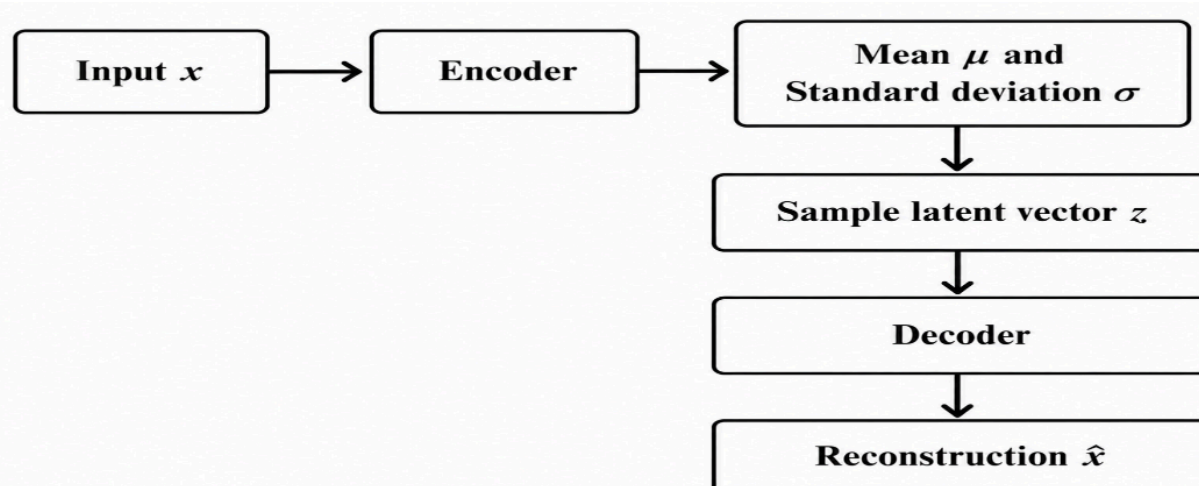
Instead, it encodes the input into a probability distribution.

For each input, the encoder generates:

- A mean vector, μ
- A variance or standard-deviation vector, σ

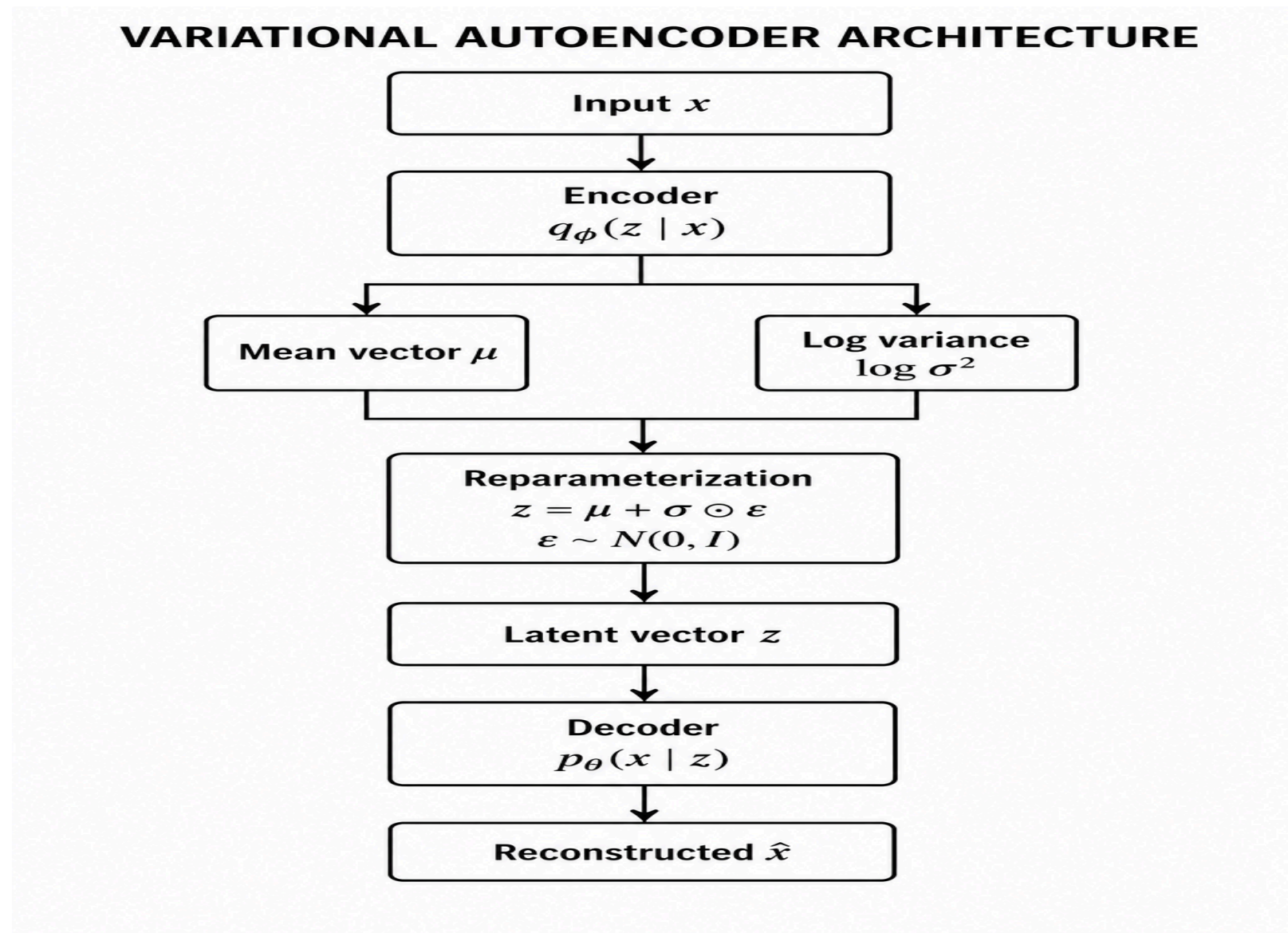
A latent vector is then sampled from this distribution.

Variational Autoencoder:



The probabilistic nature of the VAE produces a smooth, continuous, and organized latent space.

VAE Architecture



The VAE contains:

1. Encoder
2. Mean layer
3. Variance layer
4. Sampling operation
5. Latent vector
6. Decoder
7. Reconstruction-loss component
8. KL-divergence component

Probabilistic Interpretation of a VAE

A VAE assumes that the observed data is generated from hidden latent variables.

First, a latent variable is sampled from a prior distribution:

$$z \sim p(z)$$

The prior is usually a standard normal distribution:

$$p(z) = N(0, I)$$

The decoder then generates data from the latent vector:

$$x \sim p_{\theta}(x | z)$$

WORKING OF A VARIATIONAL AUTOENCODER

Step 1: Input is provided

An input sample such as an image is passed to the encoder.
Input image x will convert to a Encoder neural network

Step 2: Encoder produces distribution parameters

Instead of producing one latent vector, the encoder generates two vectors:

Mean vector $\mu = [\mu_1, \mu_2, \dots, \mu_d]$

Standard deviation $\sigma = [\sigma_1, \sigma_2, \dots, \sigma_d]$

These values define a Gaussian distribution for each latent dimension.

For example:

$\mu = [0.5, -0.2, 1.1]$

$\sigma = [0.8, 0.4, 0.6]$

This means that the input is represented not by one point but by a region in latent space.

Step 3: Latent vector is sampled

A latent vector is sampled from the distribution.

Directly sampling from a distribution would interrupt gradient-based learning. Therefore, the VAE uses the reparameterization trick.

First, random noise is sampled:

$\epsilon \sim N(0, I)$

The latent vector is then computed as:

$z = \mu + \sigma \odot \epsilon$

where $\odot$ represents element-wise multiplication.

This separates randomness from the trainable parameters.

Step 4: Decoder reconstructs the input

The sampled latent vector is passed through the decoder.

$\hat{x} = g_{\theta}(z)$

The decoder converts the latent vector into the reconstructed output.

Step 5: VAE loss is calculated

The VAE uses two loss components:

1. Reconstruction loss
2. KL-divergence loss

The total loss is:

VAE Loss = Reconstruction Loss + KL-Divergence Loss

RECONSTRUCTION LOSS IN A VAE

The reconstruction loss measures how closely the reconstructed output matches the original input.

For image data, Mean Squared Error or Binary Cross-Entropy may be used.

Conceptually:

Reconstruction Loss = Difference between x and $\hat{x}$

A small reconstruction loss means that the decoder has successfully recovered the input information.

However, using only reconstruction loss would make the VAE behave like an ordinary Autoencoder.

Therefore, a second component is required.

KL-Divergence Loss

KL divergence measures the difference between the encoder's learned latent distribution and the standard normal prior.

The encoder produces: $q\phi(z | x)$

The desired prior is: $p(z) = N(0, I)$

The KL-divergence term encourages: $q\phi(z | x) \approx p(z)$

In simple terms, it pushes the latent distributions toward a standard normal distribution.

This provides several benefits:

- Latent points become organized.
- Empty gaps are reduced.
- Nearby points produce similar outputs.
- Random sampling becomes meaningful.
- Smooth interpolation becomes possible.
- New data can be generated.

For a Gaussian latent distribution, the commonly used KL-divergence expression is:

$$KL = -\frac{1}{2} \sum (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

The mathematical details are less important than its role: it regularizes the latent space.

VAE Loss Function

The complete VAE objective can be represented as:

Total Loss = Reconstruction Loss + KL Loss

If reconstruction loss dominates:

- Reconstructed outputs may be sharp.
- Latent space may become irregular.
- Random generation may be poor.

If KL divergence dominates:

- Latent space becomes highly organized.
- Reconstructions may become blurry.
- Important input details may be lost.

EVIDENCE LOWER BOUND

The VAE is trained by maximizing a quantity called the Evidence Lower Bound, or ELBO.

$$ELBO = \text{Reconstruction quality} - \text{Latent regularization}$$

Maximizing ELBO is equivalent to minimizing the VAE loss.

The reconstruction term encourages the model to accurately reproduce the data.

The KL-divergence term encourages the learned latent distribution to remain close to the prior distribution.

Thus, ELBO balances:

- Data reconstruction
- Latent-space organization

Structure of the VAE Latent Space

An ordinary Autoencoder (AE) learns a compressed representation (latent vector) for each input sample. However, it does not impose any structure on the latent space. As a result, the encoded points may be scattered irregularly, leaving large empty regions between groups of data points.

Ordinary Autoencoder Latent Space

If a random point is selected from one of these empty gaps and decoded, the decoder has never learned to reconstruct data from that region. Consequently, it often produces

unrealistic, noisy, or invalid outputs. Therefore, a standard autoencoder is mainly useful for data reconstruction rather than generating new samples.

A Variational Autoencoder (VAE) addresses this limitation by representing each input as a probability distribution instead of a single latent point. During training, the encoder predicts the mean (μ) and variance (σ^2) of a Gaussian distribution for every input. At the same time, the latent distributions are regularized to follow a common prior distribution, typically a standard normal distribution $N(0|1)$. This regularization creates a well-organized and continuous latent space.

VAE Latent Space

Because the latent vectors are distributed smoothly, nearby points represent similar features, and there are very few empty regions. This allows the decoder to generate meaningful outputs even for points that were never directly observed during training.

The structured latent space provides several important advantages:

- Nearby latent points decode into similar samples, preserving the continuity of the learned representation.
- Intermediate points produce realistic outputs, enabling smooth interpolation between two different data samples.
- Randomly sampled latent vectors can generate new, valid data, making VAEs true generative models.
- Semantic transitions become possible, where gradual movement in the latent space results in gradual changes in the generated output (for example, changing a handwritten digit from '3' to '8' or gradually altering facial attributes such as smile or age).

Generating New Data Using a VAE

After training, the encoder is not required for data generation.

The generation process is:

Sample z from $N(0, I)$ and then Pass z through decoder and then Generate a new sample

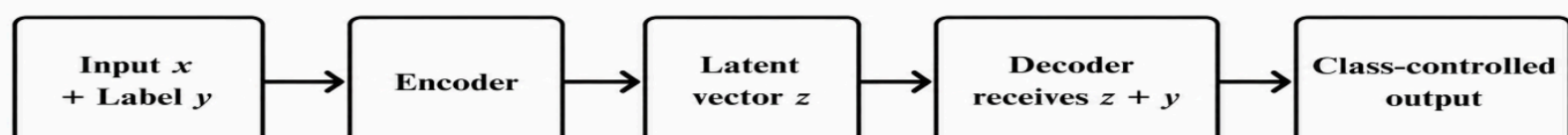
For example, a VAE trained on handwritten digits can generate new digit images.

The generated image is not necessarily an exact copy of any training image. It is a new sample created from patterns learned during training.

VARIANTS OF VARIATIONAL AUTOENCODERS

1) Conditional VAE

A Conditional VAE generates data based on additional information such as a class label.



For example, the user can request the model to generate a specific handwritten digit.

2) β -VAE

A β -VAE introduces a parameter β to control the strength of KL regularization.

$$\text{Loss} = \text{Reconstruction Loss} + \beta \times \text{KL Loss}$$

When $\beta > 1$, the model is encouraged to learn more separated or disentangled latent factors.

For a face dataset, different latent dimensions may capture:

- Pose
- Lighting
- Expression
- Hair style
- Identity-related properties

However, strong regularization may reduce reconstruction quality.

3) Convolutional VAE

A Convolutional VAE uses convolutional encoders and decoders for images. It captures spatial features more effectively than fully connected networks.

4) Sequence VAE

A Sequence VAE uses recurrent or transformer-based architectures for:

- Text generation
- Speech synthesis
- Time-series modelling
- Molecular-sequence generation

APPLICATIONS OF VAEs

Image generation

VAEs generate new images based on learned latent distributions.

Image editing

An image can be encoded, its latent vector modified, and then decoded.

Data augmentation

New synthetic samples can increase the size and diversity of a training dataset.

Anomaly detection

A VAE trained on normal data assigns low probability or high reconstruction error to abnormal samples.

Drug and molecule generation

Latent vectors can represent chemical structures, allowing new molecules to be generated.

Recommendation systems

VAEs can model hidden user preferences and generate personalized recommendations.

Missing-data completion

The learned probability distribution can estimate missing values.

Speech and music generation

VAEs can represent and generate audio features, speech patterns, and musical sequences.

19. Latent-Variable Generation

After training, new observations are generated without using the approximate posterior. A latent sample is drawn from the prior:

$$\mathbf{z} \sim p(\mathbf{z}),$$

and an observation is then sampled from the decoder distribution:

$$\tilde{x} \sim p_{\theta}(x|z).$$

GENERATIVE ADVERSARIAL NETWORKS

A Generative Adversarial Network, commonly called a GAN, is a deep-learning generative model that learns to create new data samples similar to the data available in a training dataset.

For example, a GAN trained using thousands of human-face images can generate new and realistic-looking faces that were not present in the original dataset. Similarly, GANs can generate handwritten digits, paintings, medical images, speech signals, music, and other forms of synthetic data.

GANs were introduced by Ian Goodfellow and his collaborators in 2014.

The term adversarial means opposition or competition. A GAN contains two neural networks that compete against each other during training:

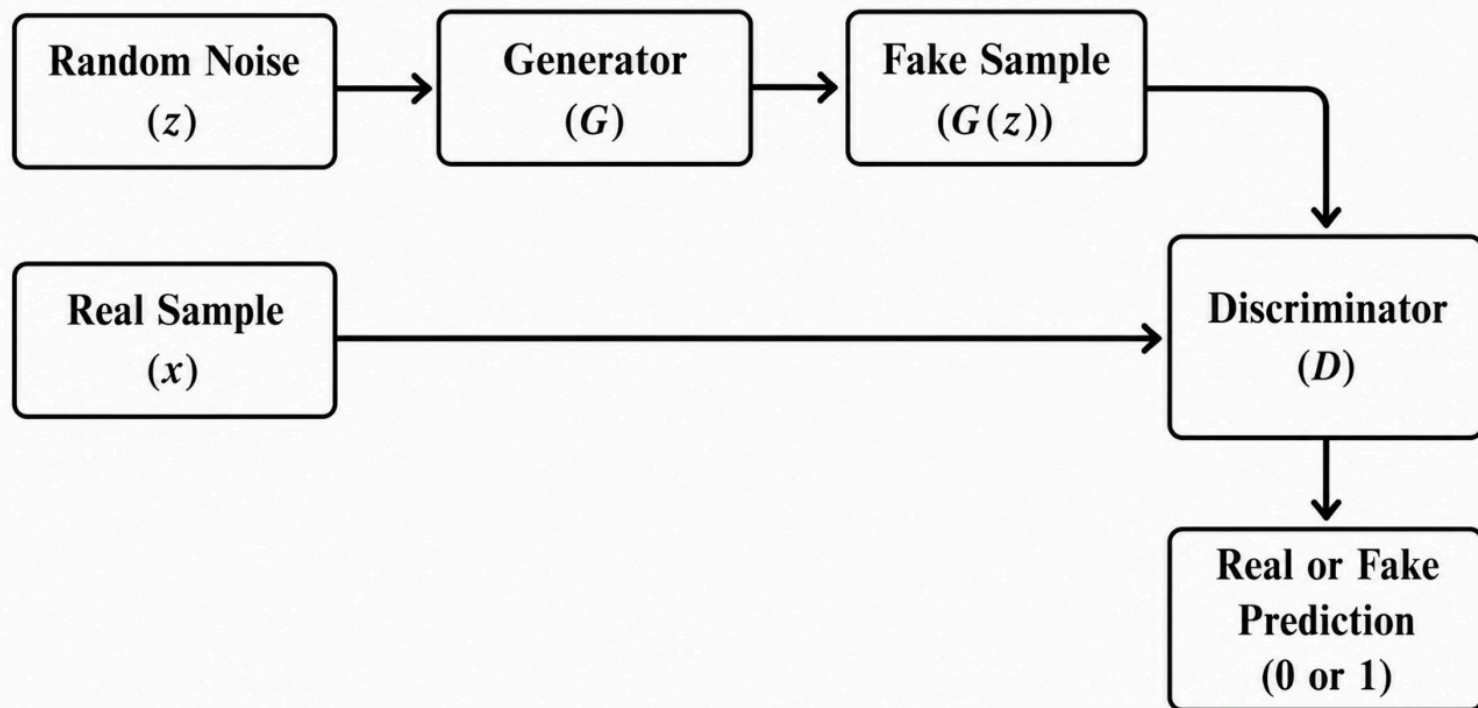
1. Generator
2. Discriminator

The Generator attempts to create realistic fake data, while the Discriminator attempts to distinguish real data from fake data. Through this competition, the Generator gradually learns to produce increasingly realistic samples.

A Generative Adversarial Network is a generative model consisting of two competing neural networks: a Generator that produces synthetic data from random noise and a Discriminator that determines whether a given sample is real or generated.

The Generator learns the distribution of the training data indirectly through feedback received from the Discriminator.

The basic process is:



The Generator attempts to fool the Discriminator, while the Discriminator attempts to avoid being fooled.

MAIN COMPONENTS OF A GAN

A GAN contains two primary components.

Generator

The Generator is a neural network that creates synthetic data.

It receives a random noise vector as its input and transforms it into a generated sample.

The Generator is represented as:

$$\hat{x} = G\theta(z)$$

where:

- z is a random noise vector.
- G is the Generator.
- θ represents the Generator parameters.
- $\hat{x}$ is the generated or fake sample.

For an image-generation GAN, the output of the Generator is a synthetic image.

The Generator does not directly receive real images as input. Instead, it learns to generate images by observing the feedback provided by the Discriminator.

Discriminator

The Discriminator is a binary classification neural network.

It receives either:

- A real sample from the training dataset, or
- A fake sample produced by the Generator.

It predicts whether the input is real or fake.

The Discriminator is represented as: $D\phi(x)$

where:

- x is an input sample.
- D is the Discriminator.
- ϕ represents the Discriminator parameters.
- The output is a probability between 0 and 1.

The output may be interpreted as:

$D(x)$ close to 1 $\rightarrow$ Sample is probably real

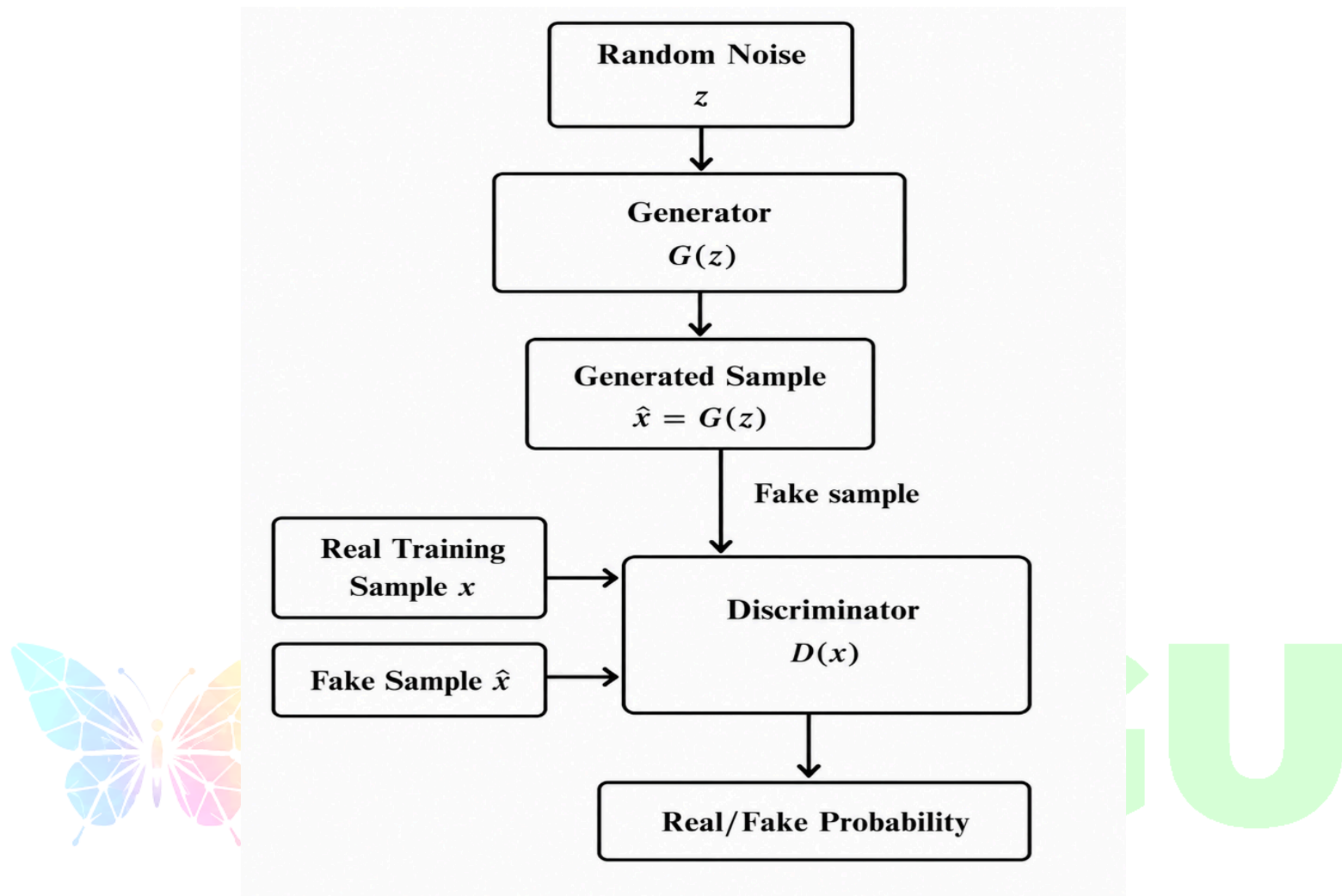
$D(x)$ close to 0 → Sample is probably fake

For example:

$D(x) = 0.95$ → 95% probability that the sample is real

$D(x) = 0.08$ → 8% probability that the sample is real

GAN ARCHITECTURE



Real sample → Expected output: 1

Fake sample → Expected output: 0

The Generator and Discriminator are trained together, but their objectives are opposite.

Generator objective: Make fake data look real.

Discriminator objective: Correctly identify real and fake data.

The input to the Generator is a random numerical vector called a noise vector, latent vector, or latent code.

It is generally represented by: z

The noise vector is sampled from a simple probability distribution such as: $z \sim N(0, I)$

or

$z \sim \text{Uniform}(-1, 1)$

A sample noise vector may look like:

$z = [0.41, -0.72, 0.18, 1.05, -0.36, \dots, 0.27]$

This vector does not initially contain recognizable information such as eyes, hair, edges, or objects. It contains randomly generated numerical values.

The Generator learns how to convert these random values into meaningful structures.

Random noise provides variation to the Generator.

If the Generator always received the same input, it would repeatedly produce the same output. Different random noise vectors allow it to generate different samples.

For example:

Noise vector $z_1 \rightarrow \text{Generator} \rightarrow \text{Face 1}$

Noise vector $z_2 \rightarrow \text{Generator} \rightarrow \text{Face 2}$

Noise vector $z_3 \rightarrow \text{Generator} \rightarrow \text{Face 3}$

All three faces may look realistic, but they will have different characteristics.

The noise vector can indirectly control features such as:

- Face shape
- Hair style
- Facial expression
- Pose
- Lighting
- Writing style
- Object colour
- Texture
- Orientation

These meanings are not manually assigned to the noise values. They are learned automatically during training.

Working of a GAN

GAN training can be divided into two alternating stages:

1. Training the Discriminator
2. Training the Generator

These two stages are repeated many times.

Step 1: Select Real Samples

A batch of real samples is selected from the training dataset.

For example, if the GAN is trained to generate faces, a batch of real face images is selected.

The desired label for real samples is: 1

The Discriminator should learn:

Real image $\rightarrow$ Output close to 1

Step 2: Generate Random Noise

Random noise vectors are sampled from a probability distribution.

For example:

$z_1 = [0.21, -0.45, \dots, 0.76]$

$z_2 = [-0.82, 0.37, \dots, 0.14]$

$z_3 = [0.61, 0.20, \dots, -0.55]$

Each noise vector acts as a starting point for creating a different generated sample.

Step 3: Generator Creates Fake Samples

The random noise vectors are passed through the Generator.

$z \rightarrow G(z) \rightarrow \text{Generated sample}$

At the start of training, the generated samples are unrealistic.

After repeated training, they become increasingly similar to the real training samples. The generated samples are assigned the label: 0 when training the Discriminator.

Step 4: Discriminator Examines Real Samples

The real samples are passed to the Discriminator.

The Discriminator predicts whether each sample is real. The correct target is 1.

If the Discriminator predicts 0.9, it has correctly recognized the sample as real.

If it predicts 0.2, it has made an incorrect prediction and its weights are updated.

Step 5: Discriminator Examines Fake Samples

The generated samples are also passed to the Discriminator.

The correct target for a fake sample is 0.

The Discriminator is trained to produce:

$D(x) \rightarrow 1$ for real samples

$D(G(z)) \rightarrow 0$ for generated samples

The Discriminator loss is calculated using both real and fake predictions.

Step 6: Update the Discriminator

During Discriminator training, only the Discriminator parameters are updated.

The Generator parameters remain unchanged.

The Discriminator learns from:

- Real samples labelled as 1
- Generated samples labelled as 0

After the update, the Discriminator becomes better at separating real and fake samples.

Step 7: Train the Generator

The Generator must learn to fool the Discriminator.

A new set of noise vectors is passed through the Generator.

The generated samples are sent to the Discriminator.

However, while training the Generator, the desired target is treated as: 1

This does not mean that the fake sample is actually real. It means that the Generator is rewarded when the Discriminator classifies the generated sample as real.

During this stage:

- Generator parameters are updated.
- Discriminator parameters are temporarily kept fixed.

The Generator receives feedback through the Discriminator and modifies its weights to produce more convincing samples.

Step 8: Repeat the Competition

ver time:

- The Discriminator becomes better at detecting fake samples.
- The Generator becomes better at producing realistic samples.
- The Generator eventually learns the major patterns of the real dataset.

Adversarial Learning

The word adversarial refers to the competitive relationship between the Generator and Discriminator.

The Generator attempts to maximize the probability that generated samples are classified as real.

The Discriminator attempts to correctly classify both real and generated samples.

This is similar to a competition between a counterfeiter and an investigator.

- The Generator behaves like a counterfeiter producing fake currency.
- The Discriminator behaves like an investigator detecting fake currency.
- As the investigator improves, the counterfeiter must produce better copies.
- As the counterfeiter improves, the investigator must identify smaller defects.

Through continuous competition, both systems improve.

13. GAN Objective Function

The original GAN objective is: $\min_G \max_D [E \log D(x) + E \log(1 - D(G(z)))]$

The objective contains two parts.

The first part is: $\log D(x)$

This encourages the Discriminator to assign a high probability to real samples.

The second part is: $\log(1 - D(G(z)))$

This encourages the Discriminator to assign a low probability to generated samples.

The Discriminator tries to maximize the complete objective.

The Generator tries to minimize the Discriminator's ability to identify generated samples.

In practical implementations, the Generator is often trained to maximize: $\log D(G(z))$

This provides stronger gradients during the early stages of training.

GAN VARIANTS

Deep Convolutional GAN

A DCGAN uses convolutional and transposed-convolutional layers.

It is widely used for image generation.

Conditional GAN

A Conditional GAN receives additional information such as a class label.

Random noise + Class label → Generator → Controlled output

For example, the model can be instructed to generate a specific digit.

Wasserstein GAN

A Wasserstein GAN uses a different training objective to improve stability and reduce mode collapse.

CycleGAN

CycleGAN performs image-to-image translation without requiring directly paired images.

Applications include:

- Horse to zebra
- Summer to winter
- Photograph to painting

StyleGAN

StyleGAN provides better control over visual characteristics and is widely known for high-quality face generation.

Pix2Pix

Pix2Pix performs paired image-to-image translation.

Applications include:

- Sketch to photograph
- Map to satellite image
- Black-and-white image to colour image

IMAGE SYNTHESIS USING DIFFUSION MODELS

Image synthesis is the process of generating new and realistic images using artificial intelligence. Unlike conventional image processing techniques that modify an existing image, image synthesis creates entirely new images that have never existed before.

Image synthesis refers to the computational process of creating a new image by learning the probability distribution of a large collection of training images. During training, the model learns common visual characteristics such as object shapes, textures, colours, lighting conditions, edges, and spatial relationships between different objects. After learning these patterns, the model can generate entirely new images that appear realistic while maintaining consistency with the learned data distribution.

For example, if a diffusion model is trained using millions of landscape images, it learns concepts such as mountains, rivers, trees, skies, shadows, and lighting conditions. When prompted with the text "A mountain landscape during sunrise", the model does not retrieve an existing image from its training data. Instead, it synthesizes a completely new image by combining the learned visual features in a meaningful way.

Thus, image synthesis can be viewed as the process of generating new visual content by using previously learned knowledge rather than reproducing stored images.

Image Synthesis Using Diffusion Models

Diffusion Models generate images by learning how to reverse a gradual noise corruption process. During training, a small amount of Gaussian noise is repeatedly added to real images until they become completely random noise. The neural network observes this process and learns how the added noise can be removed.

During image generation, the model performs the reverse operation. Instead of starting with a real image, it starts with a random noise image sampled from a Gaussian distribution. The model then removes small amounts of noise in multiple steps. As the noise decreases, meaningful structures such as edges, object boundaries, textures, colours, and lighting gradually emerge until a realistic image is obtained.

Unlike GANs, which attempt to generate the final image in a single forward pass, diffusion models generate images progressively. Each denoising step improves the quality of the image, resulting in highly detailed and visually coherent outputs.

Working Principle of Image Synthesis

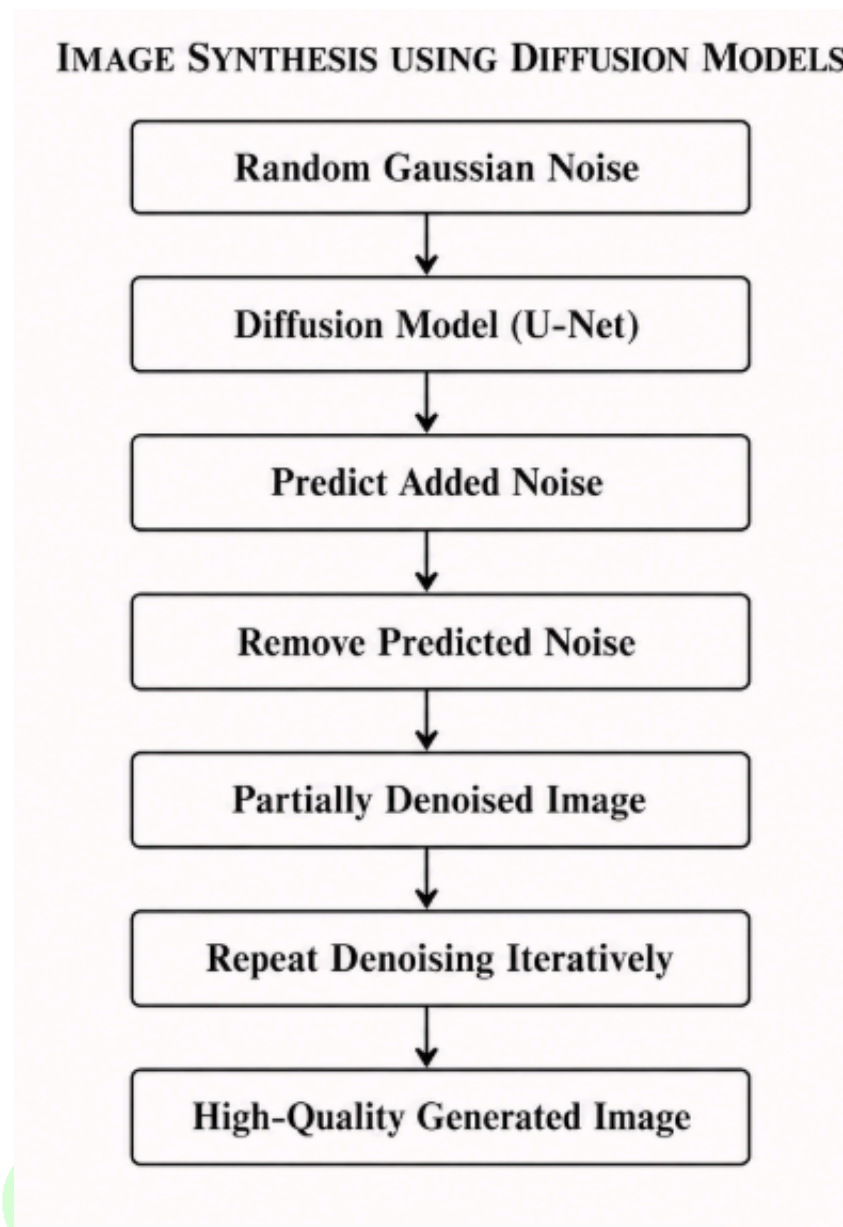
The image synthesis process consists of two major phases: training and image generation (inference).

Training Phase

During the training phase, the model learns the relationship between noisy images and their corresponding clean images. A predefined amount of Gaussian noise is added to each training image over several time steps. At every step, the neural network predicts the noise that was added. By minimizing the prediction error, the network gradually learns how to distinguish meaningful image information from random noise.

Generation phase

Once the training process is complete, the model enters the generation phase. The generation process begins with a randomly generated Gaussian noise image. The trained neural network repeatedly predicts the noise present in the image and removes it step by step. After several iterations, the random noise gradually transforms into a realistic image.



Step-by-Step Image Generation Process

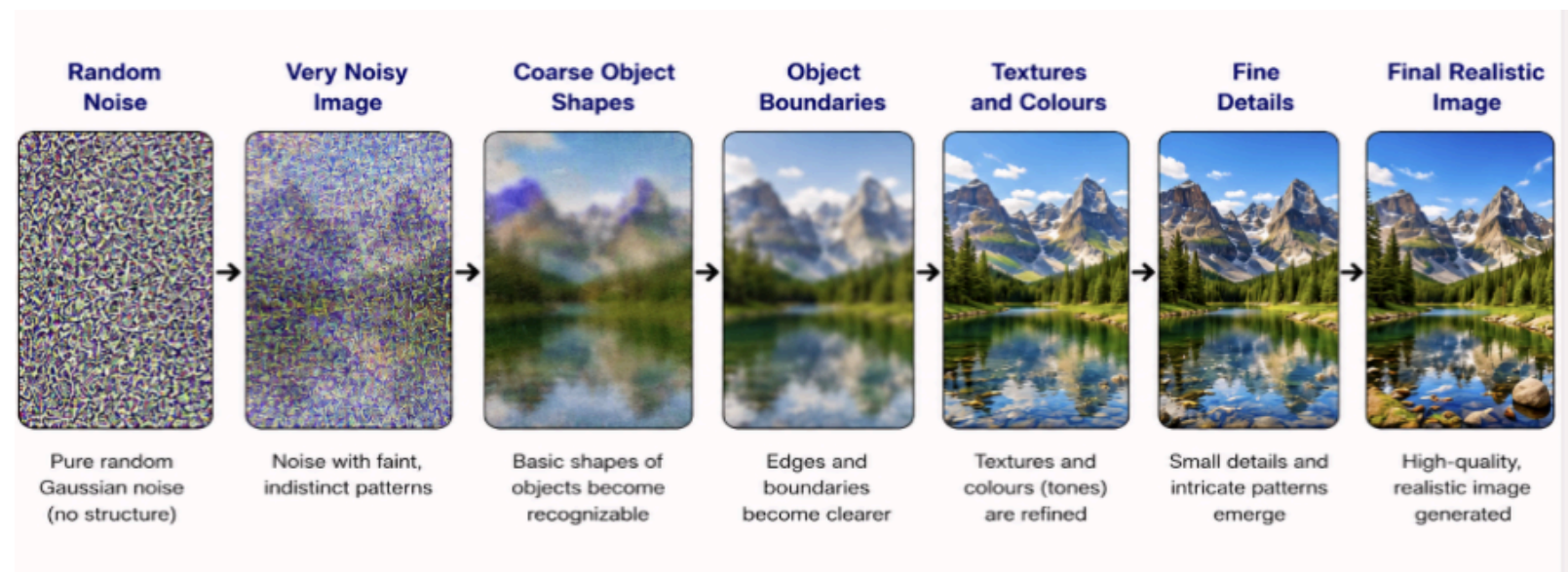
The image generation process begins with a randomly generated Gaussian noise image. At this stage, the image contains no meaningful information and appears completely random.

The noisy image is then provided as input to the trained diffusion model. The model estimates the amount of noise present and predicts which portion of the image corresponds to random noise.

The predicted noise is subsequently removed from the image. After the first denoising step, the image remains noisy but begins to exhibit coarse structural information.

This denoising procedure is repeated many times. During each iteration, the model progressively improves the image by refining edges, recovering object shapes, enhancing textures, and preserving fine details. As the number of denoising steps increases, the image gradually evolves from random noise into a realistic visual representation.

The complete process is illustrated below.



COMPARATIVE ANALYSIS OF GENERATIVE ARCHITECTURES

Autoencoder (AE)

An Autoencoder is the simplest deep generative architecture. It consists of an Encoder that compresses the input into a latent representation and a Decoder that reconstructs the original input from this compressed representation. Since its primary objective is reconstruction rather than generation, it cannot effectively create diverse new samples.

Advantages

- Simple architecture.
- Fast training and inference.
- Excellent for feature extraction and dimensionality reduction.
- High reconstruction accuracy.

Limitations

- Poor capability for generating new data.
- Discontinuous latent space.
- Generated images often lack diversity.

Variational Autoencoder (VAE)

A Variational Autoencoder extends the traditional Autoencoder by learning a probability distribution instead of fixed latent representations. Each input is represented by a mean and variance, enabling the model to sample new latent vectors and generate previously unseen images.

Compared with Autoencoders, VAEs generate more diverse images, although the outputs are often slightly blurry due to probabilistic reconstruction.

Advantages

- Continuous latent space.
- Good diversity in generated samples.
- Stable training.
- Suitable for interpolation and data generation.

Limitations

- Images are generally blurrier than GANs and Diffusion Models.
- Lower visual fidelity.

Generative Adversarial Networks (GANs)

GANs consist of two competing neural networks:

- **Generator**, which creates synthetic images.
- **Discriminator**, which distinguishes real images from generated images.

During training, both networks improve simultaneously until the Generator produces images that are difficult for the Discriminator to distinguish from real images. GANs generate highly realistic images in a single forward pass, making them computationally efficient during inference.

Advantages

- Generates sharp and realistic images.
- Fast image generation.
- Suitable for real-time applications.

Limitations

- Difficult and unstable training.
- Sensitive to hyperparameter tuning.
- May suffer from mode collapse, where the Generator repeatedly produces similar outputs.

Diffusion Models

Diffusion Models represent the latest generation of deep generative architectures. Instead of generating an image directly, they begin with random Gaussian noise and gradually remove the noise through multiple denoising steps until a realistic image is obtained. This iterative process enables the model to generate images with exceptional detail, diversity, and visual quality.

Advantages

- Stable training process.
- Extremely realistic image generation.
- High diversity with minimal risk of mode collapse.
- Excellent preservation of textures and fine details.
- Supports text-to-image, image editing, inpainting, and super-resolution.

Limitations

- Computationally intensive.
- Image generation is slower because it requires many denoising iterations.
- High memory and GPU requirements.

Parameter	Autoencoder (AE)	Variational Autoencoder (VAE)	Generative Adversarial Network (GAN)	Diffusion Model
Primary Objective	Learn compressed representations and reconstruct data	Learn latent probability distribution for data generation	Generate realistic images by using adversarial learning	Generate images by iterative denoising
Working Principle	Encoder–Decoder reconstruction	Probabilistic Encoder–Decoder with latent sampling	Competition between Generator and Discriminator	Forward noise addition and reverse denoising
Image Quality	Moderate	Good but slightly blurry	High-quality and sharp	Excellent and highly realistic
Training Stability	Stable	Stable	Difficult and unstable	Highly stable
Image Diversity	Limited	Good	Good (may suffer from)	Very High

			mode collapse)	
Generation Speed	Very Fast	Fast	Very Fast	Slow (multiple denoising steps)
Computational Cost	Low	Moderate	Moderate	High
Major Advantage	Simple architecture with good reconstruction	Continuous latent space and smooth interpolation	Generates sharp and realistic images	Produces state-of-the-art image quality with high diversity
Major Limitation	Cannot generate diverse realistic images	Images may appear blurry	Training instability and mode collapse	Slow inference and high computational requirements
Typical Applications	Feature extraction, dimensionality reduction, image compression	Data generation, anomaly detection, latent space exploration	Face generation, image synthesis, style transfer	Text-to-image generation, image editing, inpainting, super-resolution

EVOLUTION OF MULTIMODAL AI SYSTEMS

Multimodal Artificial Intelligence (Multimodal AI) is a branch of AI that enables a machine learning model to process, understand, reason, and generate information from multiple types of data (modalities) within a single unified framework. A modality refers to a specific form of data through which information is represented. Common modalities include:

- Text
- Images
- Audio
- Video
- Speech
- Sensor data
- Documents
- 3D data

Unlike traditional AI systems that process only one modality, multimodal AI combines information from different modalities to produce more intelligent and context-aware outputs.

A multimodal AI system consists of four major stages:

Step 1: Input Acquisition

The system receives data from multiple modalities such as text, images, speech, or video.

Step 2: Feature Extraction

Each modality is processed using a specialized encoder.

- Text → Language Encoder
- Image → Vision Encoder
- Audio → Audio Encoder

Each encoder converts its input into numerical feature representations (embeddings).

Step 3: Multimodal Fusion

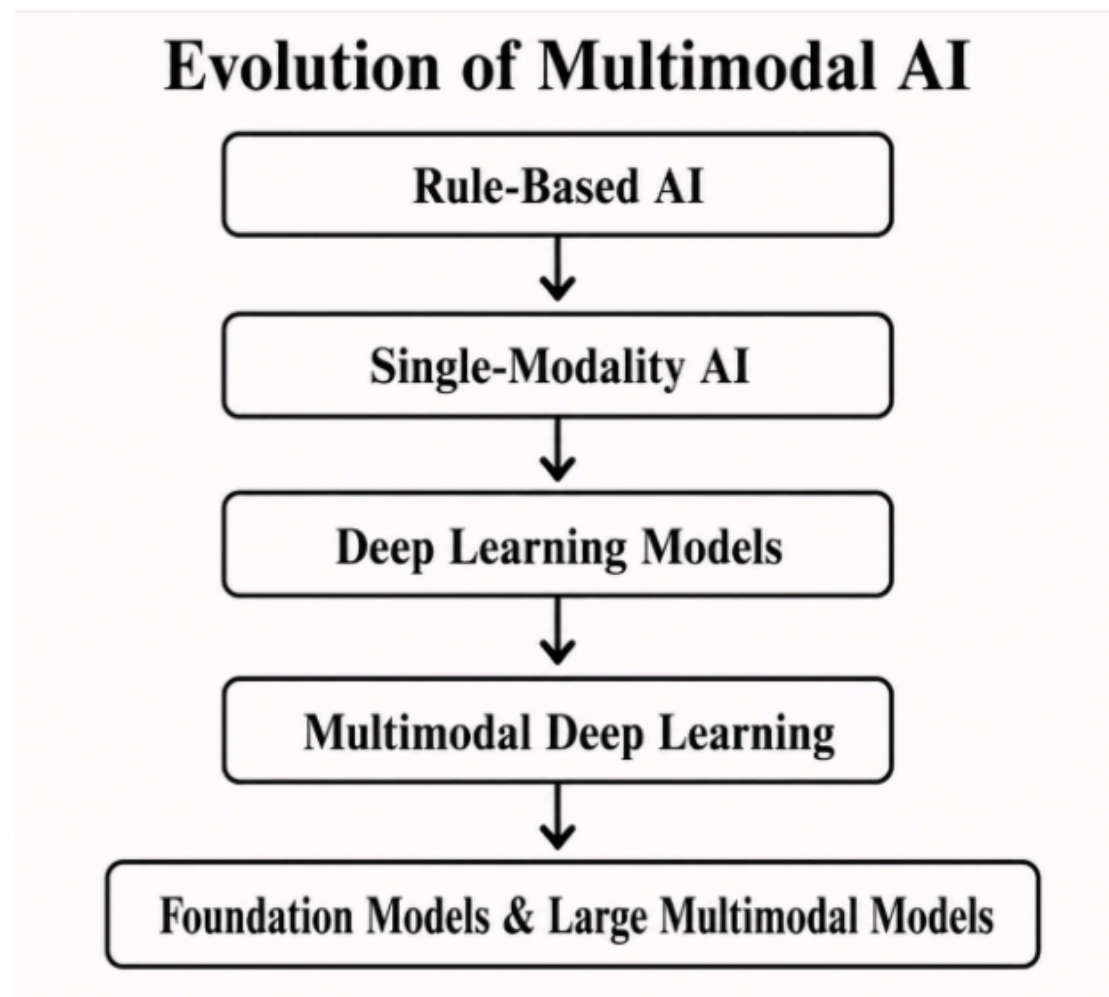
The extracted features are combined into a shared representation. During this stage, the model learns relationships between different modalities.

For example, it learns that the word "cat" corresponds to images containing a cat.

Step 4: Output Generation

The fused representation is used to generate the required output, such as:

- Text
- Image
- Caption
- Speech
- Answer
- Classification



1. Rule-Based AI Systems (1950s–1990s)

The earliest AI systems were primarily rule-based systems. These systems relied on manually written rules and logical reasoning to solve specific problems. They processed structured data and lacked the capability to learn from experience or handle unstructured information such as images or speech.

Characteristics:

- Handcrafted rules
- No learning capability
- Single input type
- Limited adaptability

Example Applications

- Expert systems
- Medical diagnosis systems
- Rule-based chatbots

2. Single-Modality AI Systems (1990s–2012)

With the emergence of machine learning and deep learning, AI systems began processing individual data modalities more effectively. Separate models were developed for different domains.

- **Natural Language Processing (NLP):** Text analysis and language

understanding.

- **Computer Vision (CV):** Image classification and object detection.
- **Speech Processing:** Speech recognition and synthesis.

Although these systems achieved excellent performance individually, they operated independently and could not combine information across modalities.

Example

- Text → Sentiment Analysis
- Image → Object Detection
- Audio → Speech Recognition

Each task required a separate model.

3. Deep Learning-Based AI (2012–2018)

The success of deep neural networks revolutionized artificial intelligence. Convolutional Neural Networks (CNNs) significantly improved image understanding, while Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks enhanced sequence modeling and speech processing.

During this stage, researchers began combining different neural networks to process multiple input types. Although these models represented an initial step toward multimodal AI, the integration between modalities remained limited.

Key Technologies

- CNN
- RNN
- LSTM
- Autoencoders
- GANs

4. Multimodal Deep Learning (2018–2022)

The introduction of the Transformer architecture transformed multimodal learning. Instead of designing separate networks for each modality, researchers developed models capable of jointly learning relationships between text, images, audio, and video.

These models learn a shared feature representation where information from different modalities is aligned within a common embedding space. As a result, the model can associate textual descriptions with corresponding visual content.

Examples include:

- Image Captioning
- Visual Question Answering (VQA)
- Speech-to-Text Translation
- Image Retrieval using Text Queries

Representative Models:

- CLIP
- ALIGN
- ViLBERT
- VisualBERT

5. Foundation Models and Large Multimodal Models (2022–Present)

The latest stage in the evolution of AI involves **Large Multimodal Models (LMMs)**. These models are trained on massive multimodal datasets containing text, images, audio, video, and code. Unlike earlier systems, they can understand and generate multiple types of content using a single unified architecture.

Modern multimodal foundation models combine language models with vision encoders and other modality-specific components, enabling them to perform a wide range of

tasks without requiring separate models for each application.

Examples include:

- GPT-4 Vision
- Gemini
- Claude (Vision)
- LLaVA
- Qwen-VL

Advantages of Multimodal AI

The evolution toward multimodal AI has provided several important advantages:

- Improved contextual understanding by combining multiple information sources.
- Better reasoning through cross-modal relationships.
- Enhanced performance in complex real-world tasks.
- Reduced need for multiple independent AI models.
- Ability to understand and generate different forms of content within a single framework.
- Greater flexibility across diverse application domains.

1. Text Understanding

Text Understanding is the ability of an AI system to read, interpret, analyze, and generate human language. It is one of the fundamental tasks of Natural Language Processing (NLP) and enables AI systems to understand the meaning, context, grammar, sentiment, and intent of textual information.

Modern language models represent words and sentences as numerical vectors called embeddings, allowing the model to identify semantic relationships between different words and phrases.

How Text Understanding Works

The text understanding process typically consists of the following stages:

1. Input Text
2. Tokenization
3. Embedding Generation
4. Language Understanding
5. Contextual Reasoning
6. Output Generation

Applications of Text Understanding

- Machine Translation
- Sentiment Analysis
- Text Summarization
- Question Answering
- Chatbots
- Information Retrieval
- Document Classification

2. Image Understanding

Image Understanding is the ability of an AI system to analyze visual information and extract meaningful knowledge from images. Unlike simple image classification, image understanding involves recognizing objects, identifying relationships among objects, understanding scenes, interpreting actions, and answering questions about visual content.

Computer Vision models convert images into numerical feature representations that capture shapes, textures, colours, edges, and spatial relationships.

How Image Understanding Works

The image understanding process consists of the following stages:

1. Input Image
2. Image Preprocessing
3. Feature Extraction
4. Object Recognition
5. Scene Understanding
6. Output Generation

Applications of Image Understanding

- Object Detection
- Face Recognition
- Medical Image Analysis
- Autonomous Driving
- Image Captioning
- Satellite Image Analysis
- Industrial Inspection

3. Audio Understanding

Audio Understanding is the ability of an AI system to analyze sound signals and extract meaningful information from them. The input may consist of speech, music, environmental sounds, or other acoustic signals.

Unlike speech recognition, which only converts speech into text, audio understanding also identifies emotions, speakers, background sounds, and acoustic events.

How Audio Understanding Works

The audio understanding process generally consists of the following stages:

1. Input Audio
2. Signal Processing
3. Feature Extraction
4. Audio Encoder
5. Pattern Recognition
6. Output Generation

Applications of Audio Understanding

- Speech Recognition
- Speaker Identification
- Emotion Recognition
- Music Recommendation
- Voice Assistants
- Environmental Sound Detection
- Healthcare Monitoring

EMERGING MULTIMODAL FOUNDATION MODELS

1. GPT-4 Vision

GPT-4 Vision extends the capabilities of Large Language Models by integrating visual understanding. It can interpret images, diagrams, handwritten notes, graphs, and documents while generating detailed textual explanations.

Key Features

- Image understanding
- Visual reasoning
- Document analysis
- OCR with reasoning

- Chart interpretation
- Multimodal conversation

2. Gemini

Gemini is a multimodal AI model capable of processing text, images, audio, video, code, and structured data within a unified architecture. It is designed for complex reasoning and supports long-context understanding.

Key Features

- Native multimodal reasoning
- Long-context processing
- Code generation
- Video understanding
- Audio processing
- Scientific reasoning

3. Claude Vision

Claude Vision combines advanced language understanding with image analysis. It is particularly effective for document interpretation, chart analysis, and visual reasoning.

Key Features

- Image interpretation
- Document analysis
- Visual question answering
- Mathematical reasoning
- Diagram understanding

4. LLaVA (Large Language and Vision Assistant)

LLaVA integrates vision encoders with large language models to perform image-based conversations and visual reasoning tasks.

Key Features

- Image captioning
- Visual dialogue
- Scene understanding
- Object recognition
- Educational assistance

5. Qwen-VL

Qwen-VL is a multimodal vision-language model capable of understanding documents, images, and charts while supporting multilingual interactions.

Key Features

- Multilingual support
- OCR
- Document understanding
- Chart interpretation
- Visual question answering